Cognitive Systems: Neural Networks and Applied AI

Exercise 3
Spiking Neural Networks

Etienne Müller, Javier López Randulfe, Tobias Duswald

1. Summary of preceding exercises

Tobias Duswald, M.Sc.

Previously on Cognitive Systems...

Biological Neural Networks

- What is a neuron?
- What does it look like?
- How does it work?
- How can we model it?
- How successful have scientist been with different models?
- How can information be encoded in spikes?

Learning

- How can we understand learning?
- How can we understand the brain structure and architecture?
- What are different approaches to model the learning behavior?

Agenda

1. Introduction to SNN simulation with Brian2
2. Encoding (recap & hands-on)
3. A simple network for adding numbers
4. Neuromorphic hardware

1. Introduction to SNN simulation with Brian2

Tobias Duswald, M.Sc.

Different SNN simulators

GeNN

nest ::

BRIAN

TensorFlow

PyNN

PyTorch

NEURON

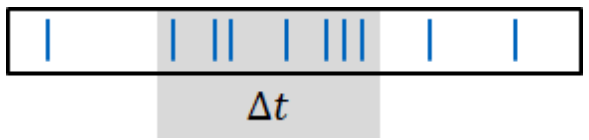
2. Encoding (recap & hands-on)

Javier López Randulfe, M.Sc.

Neural Codes

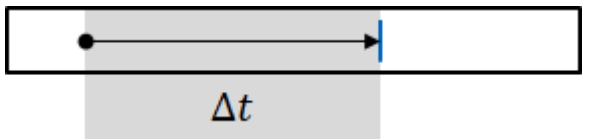
Rate Coding

The information is carried in the averaged spike rate within a certain time window (e.g. muscle excitation).



Time-to-First Spike Coding

The information is encoded in the time of emission of the first spike after a reference point (e.g. onset of a stimulus).



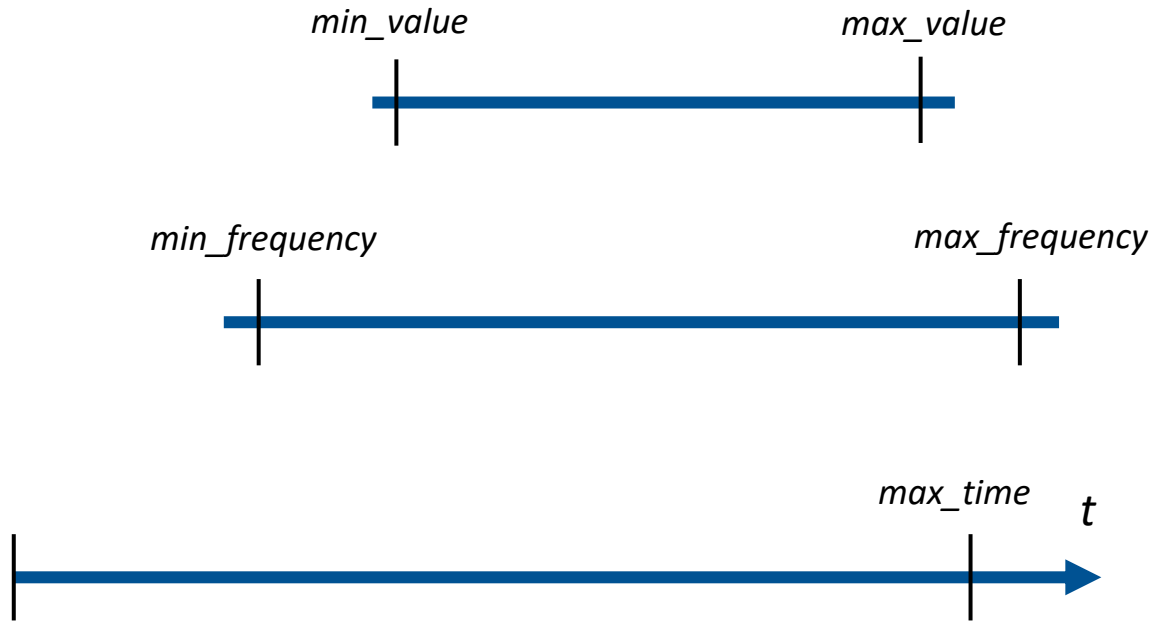
Code sample: Frequency encoding

Task: Create a function that translates floating values into spike trains with constant frequency. The frequency shall be proportional to the input value

- Input range: From *min_value* to *max_value*
- Output range: From *min_frequency* to *max_frequency*
- Initial offset *t_0*. in the NEST simulator a spike cannot occur at time *t=0*

1. Initialize function parameters

```
min_frequency = 0.03 # kHz
max_frequency = 2    # kHz
min_value = 0
max_value = 10
max_time = 100       # ms
t_0 = 0.1            # ms
```



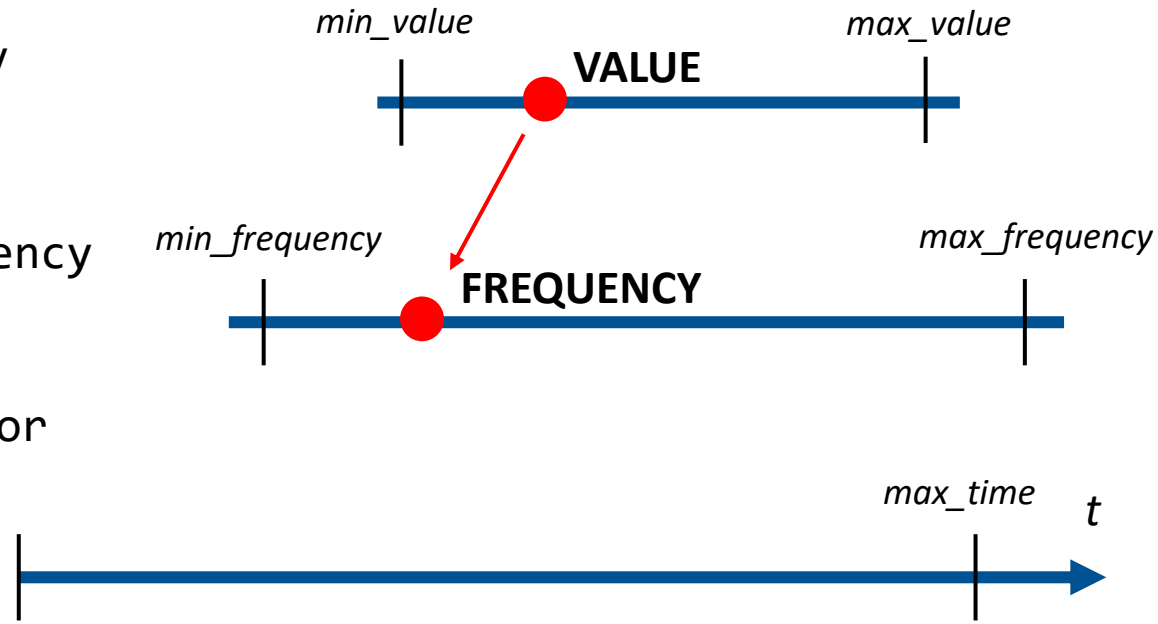
Code sample: Frequency encoding

Task: Create a function that translates floating values into spike trains with constant frequency. The frequency shall be proportional to the input value

- Input range: From *min_value* to *max_value*
- Output range: From *min_frequency* to *max_frequency*
- Initial offset *t_0*. in the NEST simulator a spike can not occur at time *t=0*

2. Perform a scale conversion, from value to frequency

```
value = 3
input_range = max_value - min_value
frequency_range = max_frequency - min_frequency
scale_factor = frequency_range / input_range
frequency = (min_frequency
              + (value-min_value)*scale_factor
             )
```



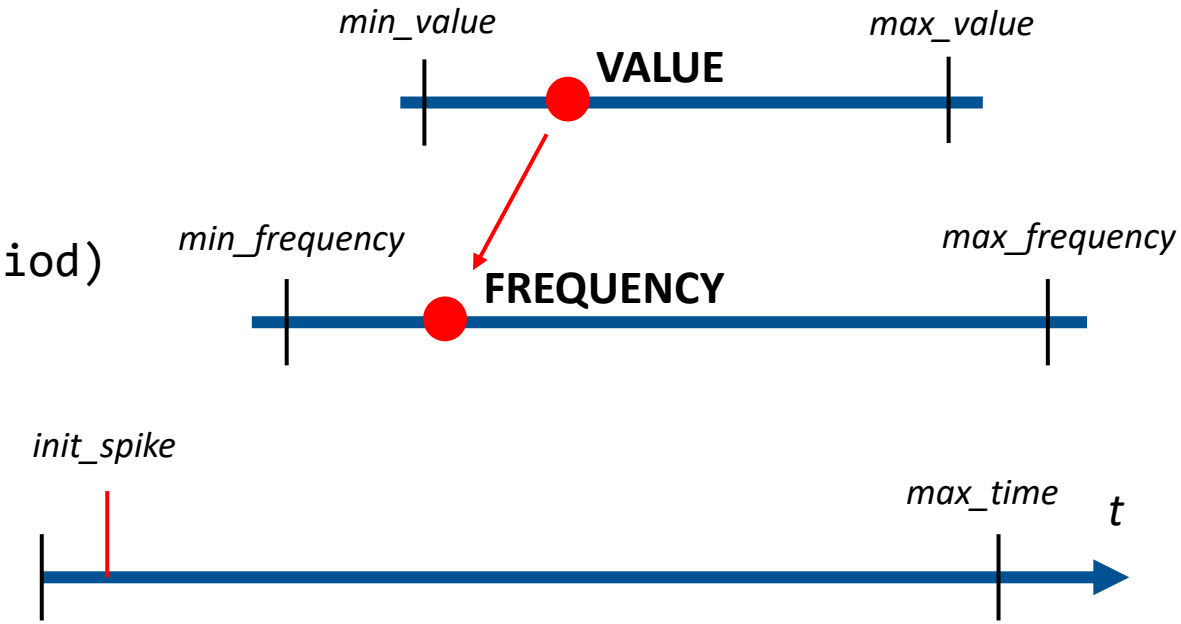
Code sample: Frequency encoding

Task: Create a function that translates floating values into spike trains with constant frequency. The frequency shall be proportional to the input value

- Input range: From *min_value* to *max_value*
- Output range: From *min_frequency* to *max_frequency*
- Initial offset *t_0*. in the NEST simulator a spike can not occur at time *t=0*

3. Calculate the period, and determine the time of the first spike with a uniform distribution

```
period = 1 / frequency
init_spike = np.random.uniform(t_0, t_0+period)
```



Code sample: Frequency encoding

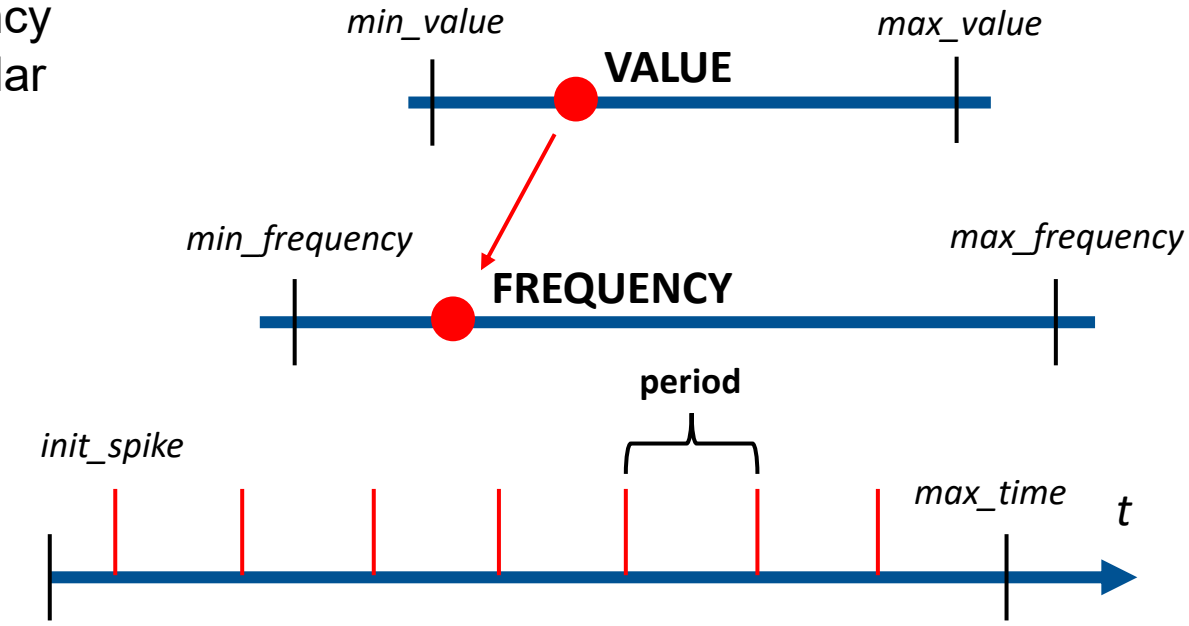
Task: Create a function that translates floating values into spike trains with constant frequency. The frequency shall be proportional to the input value

- Input range: From *min_value* to *max_value*
- Output range: From *min_frequency* to *max_frequency*
- Initial offset *t_0*. in the NEST simulator a spike can not occur at time *t=0*

4. Get the total number of spikes for the given frequency and time range, and calculate the spike times as regular time steps based on the obtained period

```
n_spikes = math.trunc(
        (max_time-init_spike)*frequency + 1
)

spike_train = np.arange(n_spikes)*period
                + init_spike
spike_train = np.around(result, 1)
```



3. A simple network for adding numbers

Javier López Randulfe, M.Sc.

Code sample: Simple NEST simulation

Task: Implement a 2-layer network that works as an integer adder

- N input integers
- 4 output neurons. Neuron j spikes if $\sum_{i}^{N} I_i \geq j$

1. Initialize NEST and network parameters

```
import nest
N_input = 4
N_output = 4
single_neuron_weights = [400, 200, 150, 100]
network_weights = [single_neuron_weights] * n_input
network_weights = list(map(list, zip(*network_weights)))
```

Code sample: Simple NEST simulation

Task: Implement a 2-layer network that works as an integer adder

- N input integers
- 4 output neurons. Neuron j spikes if $\sum_{i}^{N} I_i \geq j$

2. Create network structure

```
input_layer = nest.Create("spike_generator", n_input)
output_layer = nest.Create("iaf_psc_alpha", n_output)
nest.Connect(input_layer, output_layer, conn_spec="all_to_all",
             syn_spec={"weight":network_weights, "delay":1.0})
```

Code sample: Simple NEST simulation

Task: Implement a 2-layer network that works as an integer adder

- N input integers
- 4 output neurons. Neuron j spikes if $\sum_{i}^{N} I_i \geq j$

3. Translate input values into spikes, and feed them to the input layer

```
spike_times = [encoding.value2frequency(value, max_time=sim_time)
               for value in values]
nest.SetStatus(input_layer, "spike_times", spike_times)
```

Code sample: Simple NEST simulation

Task: Implement a 2-layer network that works as an integer adder

- N input integers
- 4 output neurons. Neuron j spikes if $\sum_{i}^{N} I_i \geq j$

4. Instantiate measuring objects for saving the spike and voltage history

```
multimeter = nest.Create("multimeter", n_output,
                         params={"record_from":["V_m"]})
nest.Connect(multimeter, output_layer, "one_to_one")
spike_detectors = nest.Create("spike_detector", n_output)
nest.Connect(output_layer, spike_detectors, "one_to_one")
```

Code sample: Simple NEST simulation

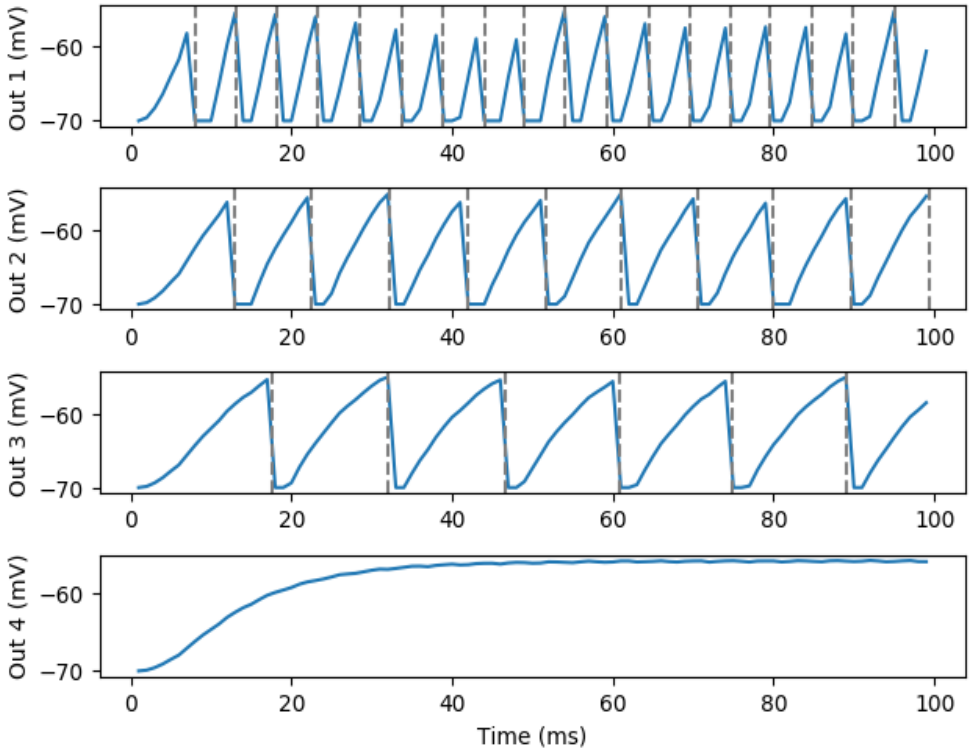
Task: Implement a 2-layer network that works as an integer adder

- N input integers
- 4 output neurons. Neuron j spikes if $\sum_{i}^{N} I_i \geq j$

5. Simulate and plot

```
nest.Simulate(sim_time)
data_analysis.print_frequency(spike_detectors)
data_analysis.plot(multimeter, spike_detectors)
```

I = [0, 1, 2, 0]



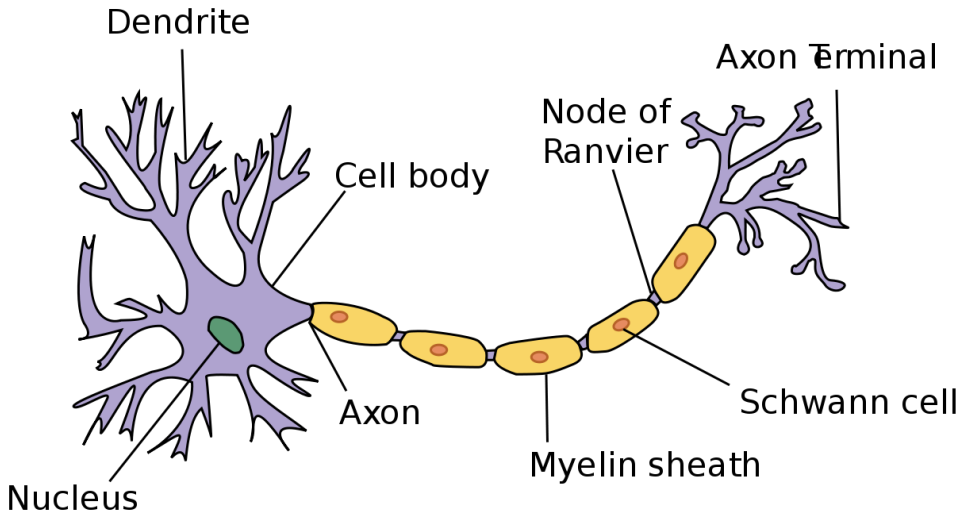
4. Neuromorphic hardware

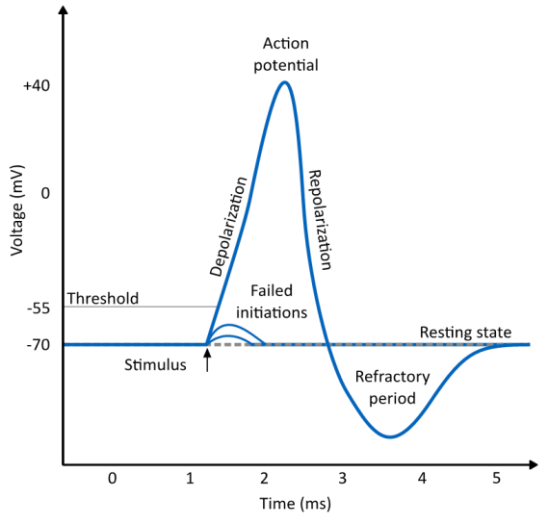
Etienne Müller, M.Sc.

Why do we need SNNs?

- Simulation of spiking neurons computationally expensive
- ANNs can already be super-human

Recap: Neurons





Dendrites: The dendrites receive electrical impulses from other neurons through synapses.

Soma: The soma is the cell body containing the nucleus and creates action potentials from stimulus.

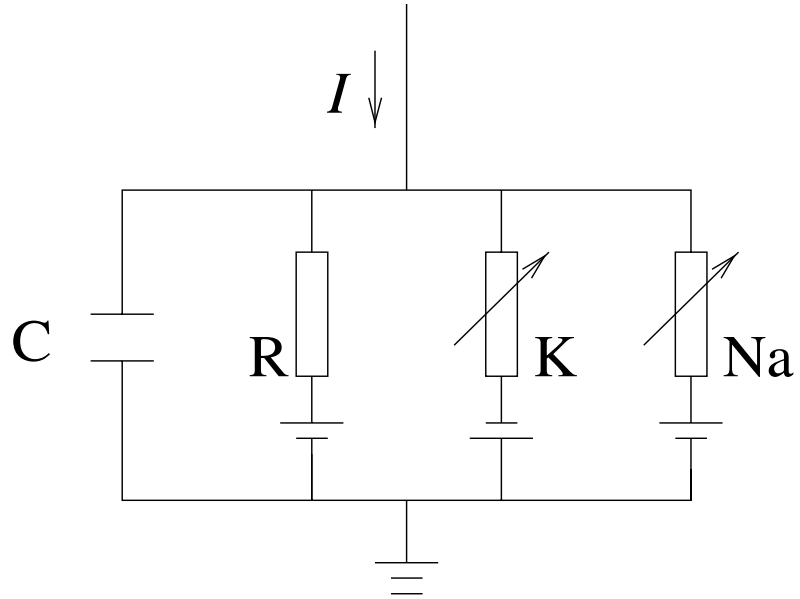
Axon: The axon emerges from the soma at the axon hillock and conducts electrical impulses to other neurons.

Action potential:

- An input current increases the membrane potential (depolarization)
- At a threshold the membrane voltage quickly increases to a peak voltage, the action potential
- After some delay the membrane voltage falls back below the resting state (hyperpolarization)

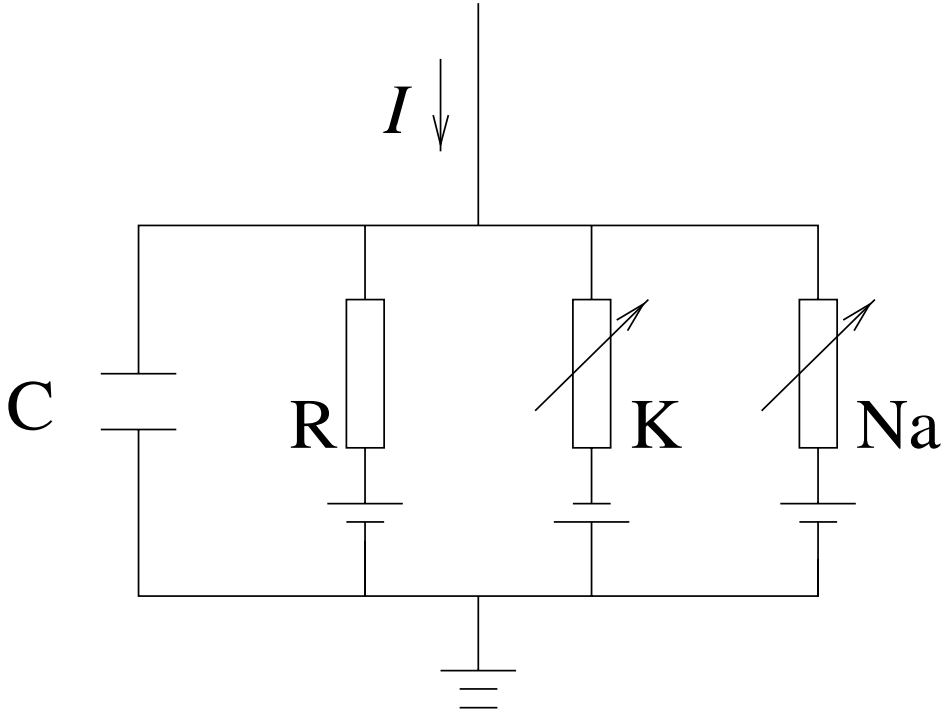
Recap: Hodgkin-Huxley Model

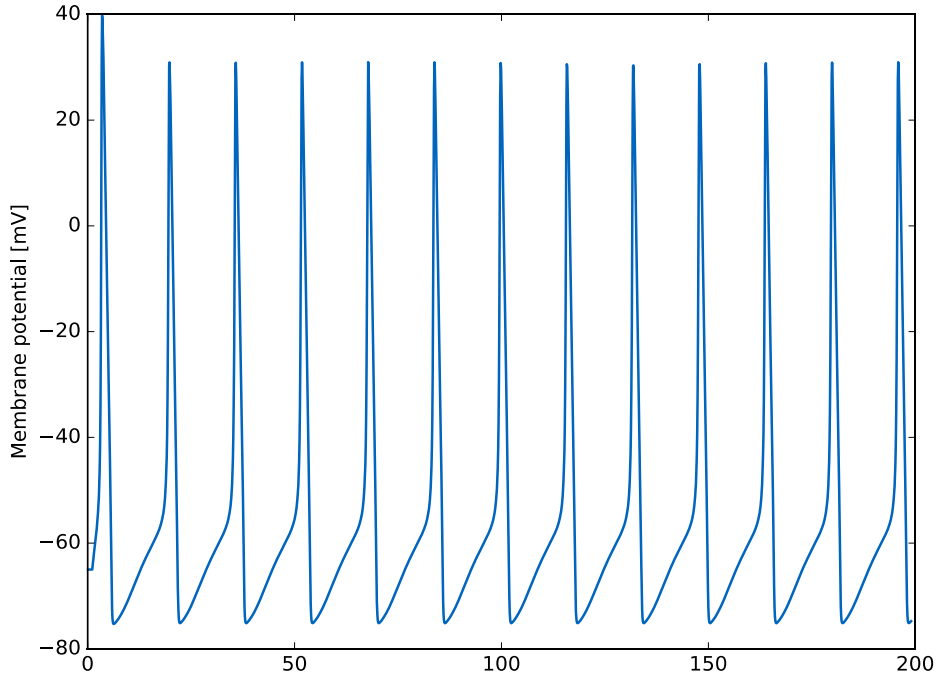
The Hodgkin-Huxley model was derived based on measurements of action potentials in the giant axon of the squid in 1952. The model contains a single compartment and is the basis for conductance-based neuron models. The overall dynamic behavior is described by the following electrical circuit:



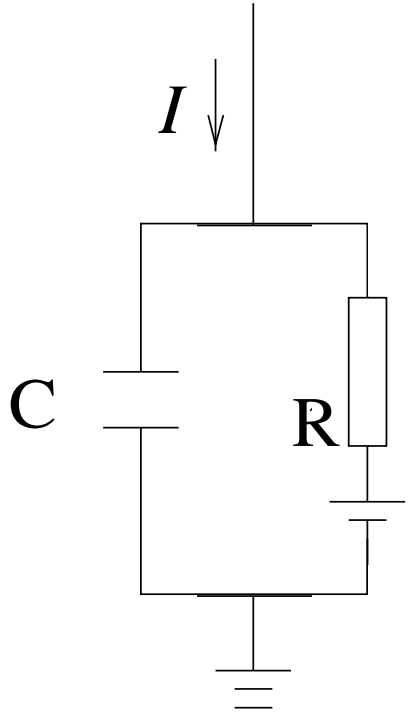
From W. Gerstner and W. M. Kistler, Spiking neuron models: Single neurons, populations, plasticity. Cambridge U.K., New York: Cambridge University Press, 2002.

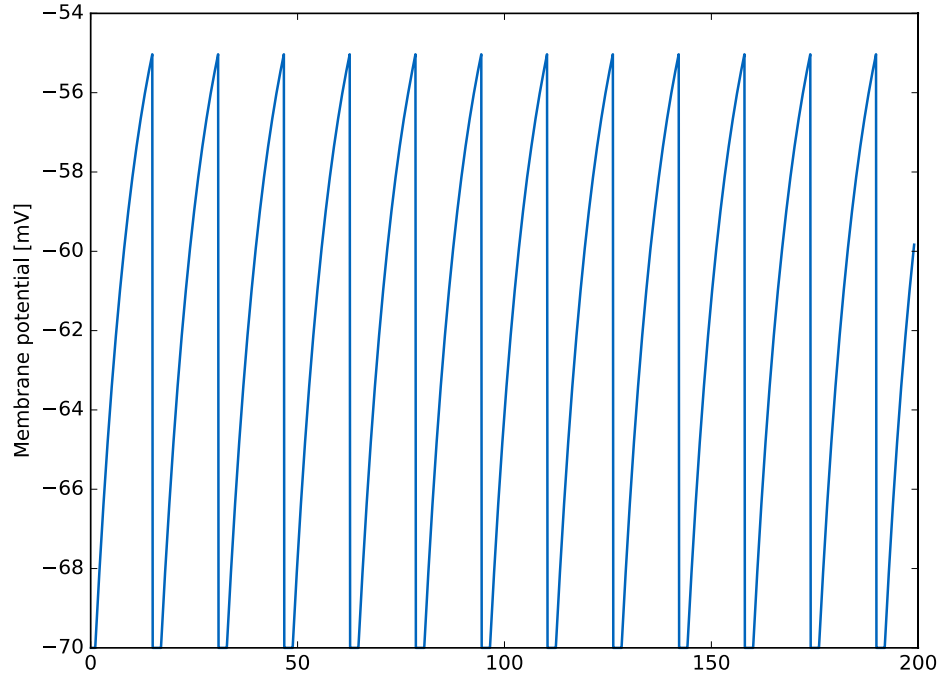
Hodgkin-Huxley Model



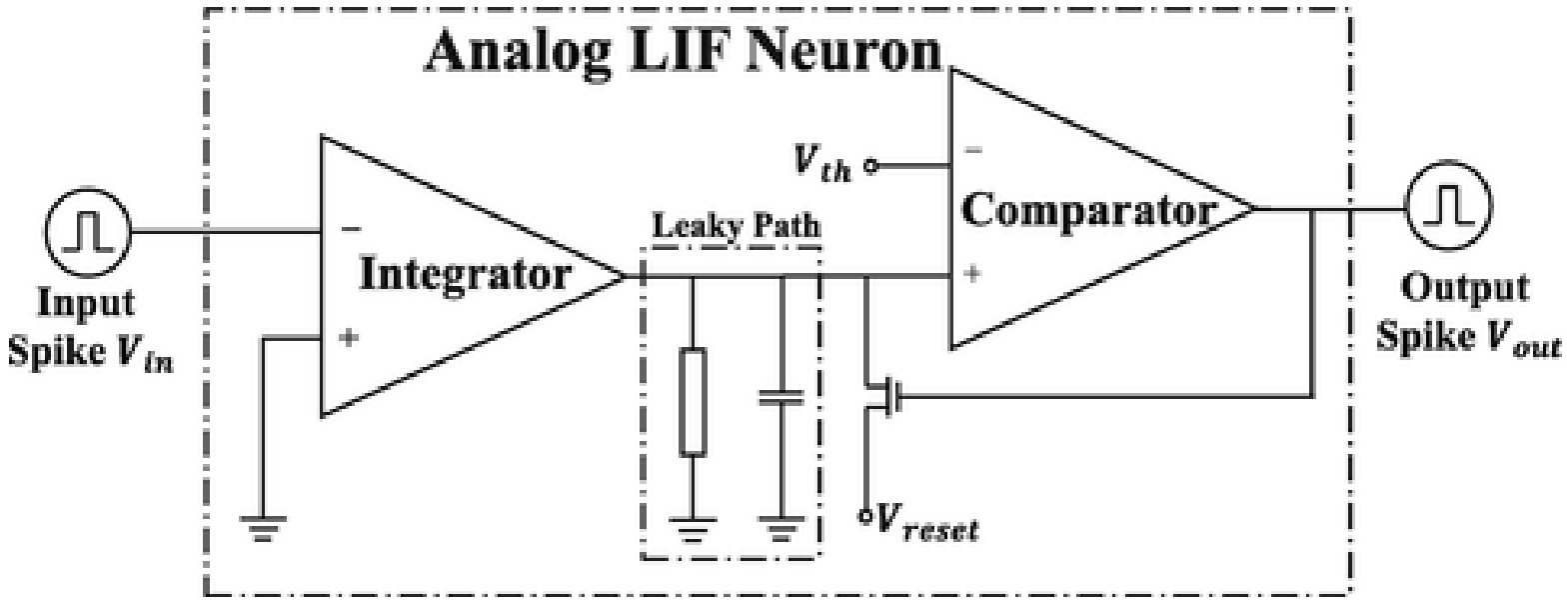


Leaky Integrate-and-Fire Model





Analog LIF Neuron



Wang et al. (2017) Energy Efficient Spiking Neural Network Design with RRAM Devices

Simulating Neurons

- SpiNNaker used for brain simulations at Human Brain Project
- For simplicity and cost reasons off-the-shelf components
- ARM9 core at 200 MHz
- Simulates 1000 neurons at 1ms timestep

Available Hardware

SpiNNaker

- Per Chip: 18 ARM9 cores, simulating 16K neurons, 16M synapses
- Final system: 460M neurons, 460B synapses

IBM TrueNorth

- 4096 cores, simulating 16M neurons, 4B synapses

Intel Loihi

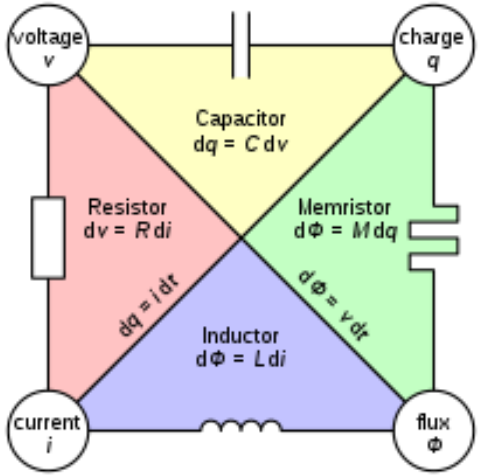
- 128 neuromorphic cores, 131K LIF neurons, 130M synapses

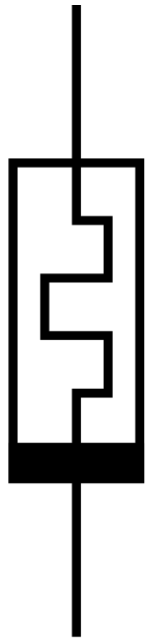
Other:

Neurogrid & Braindrop (Stanford), BrainScaleS 1 & 2 (Heidelberg), Rolls (Uni Zurich & ETH Zurich), DYNAP-SE (Uni Zurich & Lab of Giacomo Indiveri), Odin (Louvain)

Memristor

- (Memory + Resistor)
- •Resistance based on the previous amount of charge
- •Theorized by Chua in 1971 as fourth elemental two-terminal circuit, following resistor, capacitor, inductor
- •First practicel device in 2008 by HP Labs
- •High endurance (>120B cycles)
- •Long retention (>10 years)
- Active field of research, many innovative 1D and 2D materials[1]





[1] Sangwan, Hersam (2020) Neuromorphic nanoelectronic materials

Energy Efficiency

In Theory[1]:

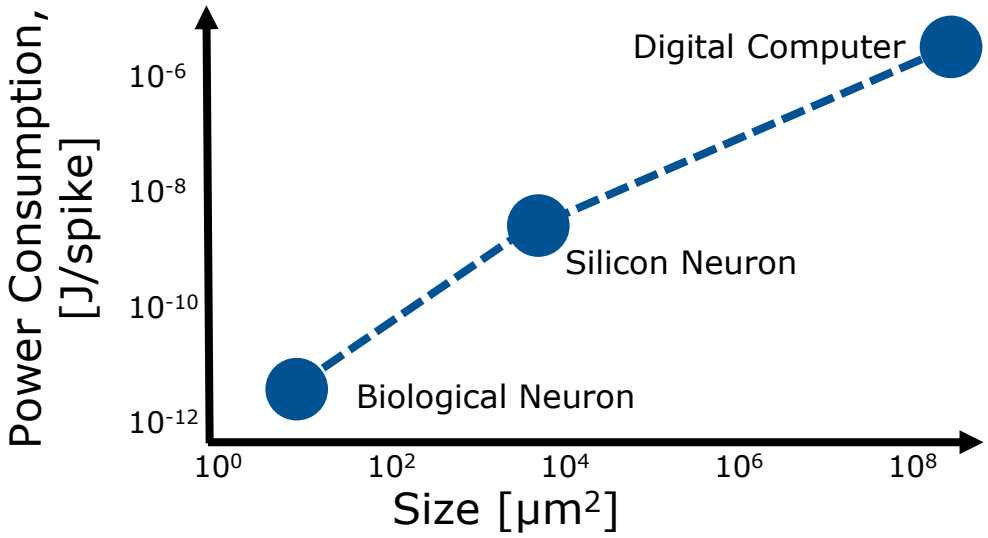
- Energy costs of biological neuron: 10^{-11} J
- Silicone Neurons: 10^{-8} J
- Digital Computer: 10^{-3}-10^{-7} J

→ 10-10,000x

In practice[2]:

- Intel Loihi vs. Intel Neural Compute Stick

→ 4.11x



[1] C.-S. Poon (2011) Neuromorphic silicon neurons and large-scale neural networks: challenges and opportunities
[2] Blouw et al. (2020) Event-Driven Signal Processing with Neuromorphic Computing Systems